



RESEARCH PROPOSAL · MECHANISTIC FOLLOW-UP STUDY

Opening the Black Box of Obfuscated-Code Comprehension

Two instruments on one Block-Model frame: **attention reallocation** (where the compute budget goes) and **Natural Language Autoencoders** (what the activations actually say).

Builds directly on **Paper 1** human comprehension under obfuscation (n = 50) · **Paper 2** LLM–human alignment (28,750 runs) · **Paper 3** dual-process signatures (n = 73)

Hypotheses H1–H4 on stacked obfuscation · Schulte's Block Model as the shared coordinate system · July 2026

The Evidence Base: Behavior Is Already Mapped

Converging findings from three prior studies — at the outcome level and at the mechanism-proxy level.

$\rho = 0.30\text{--}0.47$

Reasoning-tuned models track human task-level difficulty ($p < .001$); coder and instruct models sit near zero.

OR = 0.57

Only control-flow (L2) carries a reliable accuracy penalty; renaming's pooled effect cancels by item route.

$r = -0.196$

Dispatcher state count predicts failure under control-flow flattening ($q = 3.1 \times 10^{-23}$) — a relational bottleneck.

$\times 3.22$

Renaming triples “confused-by-names, then traced” — Stroop-like interference, not a confident trap (HCI OR 0.85, n.s.).

21.3% / 10.0%

At L1b accuracy collapses and HCI peaks only where semantic displacement $\cap$ identifier spikes co-occur — atom-level failure.

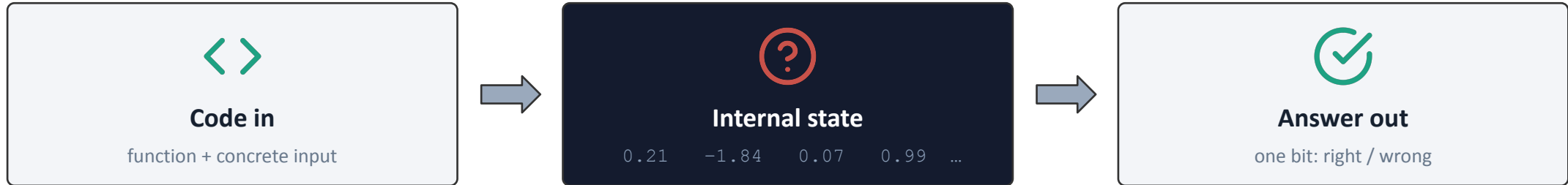
$\sim 2,048$ tok

Model System-2 accuracy saturates: a bounded reasoning budget, the machine analog of the human cognitive limit.

Bottom line We know **which** tasks break humans and models, and we know the two failure routes (atom-level interference vs. relational overload). We do not know **how** the model's internals implement — or fail to implement — either route.

The Gap: We See the Answer, Not the Understanding

A model reads code and produces an answer. In between, its computation is a stream of vectors that no measure we have taken inspects directly.



Missing readout 1 — WHERE

Attention allocation over code tokens: how a finite budget splits between identifiers, dispatcher structure, and sinks — and how that split moves from L0 to L3. This is the H1–H4 hypothesis space (Part II).



Missing readout 2 — WHAT

Representation content: what the residual stream encodes while reading obfuscated code — does the model recover the true function, adopt the adversarial frame, or silently flag the deception? NLAs answer in plain language (Part III).

Why stacked obfuscation (L3) is the decisive test. L3 attacks the atom level (misleading identifiers) and the relational level (dispatcher indirection) at once, so the two documented failure routes compete for one finite budget. Which signal the model privileges — identifiers, structure, both, or neither — is invisible to accuracy, CoT length, or confidence.

Why Obfuscated Code Is the Ideal Testbed

Obfuscation preserves program behavior while stripping the cues a reader relies on — manufacturing ambiguity, traps, and hidden state on demand.



L1

Rename to nonsense

Strip meaningful names. The model loses its lexical shortcuts and must reason about structure instead — an atom-level cue removal.



L1b

Adversarial renaming

Swap in plausible-but-wrong names. A Stroop-like trap that baits confident, incorrect answers — an atom-level cue corruption.



L2

Control-flow flattening

Scramble execution order behind a dispatcher. Forces hidden state tracking — a relational and macro-level break.

OBFUSCATION LADDER USED THROUGHOUT

L0 original

L1 renaming

L1b adversarial

L2 flattening

L3 stacked

These are precisely the conditions where an internal window is most useful — and where any explanation is most likely to crack. **Either outcome is a result.**



PART I · THEORETICAL FOUNDATION

The Block Model, In Full

Schulte's 4×3 educational model of program comprehension — the coordinate system that every measurement in this proposal maps onto, for humans and machines alike.

The Block Model: 4 Levels × 3 Dimensions

Schulte (ICER '08): every level crosses three dimensions of understanding, giving twelve comprehension cells — not a single ladder.

Level \ Dimension	Text surface (structure)	Program execution (data & control flow)	Function (purpose / goals)
Macro structure the program as a whole	Overall layout and organisation of the source text	Understanding the algorithm’s full execution behaviour	The goal of the program in its context
Relations connections between blocks	References between blocks (calls, includes)	Control and data flow across blocks; sequence of calls	How subgoals connect to goals; how functions compose
Blocks regions that belong together	Recognising a region of interest (loop body, method) syntactically	Operation of a block — what a loop or method does when run	The purpose of a block — its role in the plan
Atoms language elements	Keywords, identifiers, operators as text	Operational semantics of a single statement	Purpose of a single element — what an identifier is for



Duality of structure & function

The same code is simultaneously a static text, a dynamic execution, and a purposeful plan. Comprehension means building bridges across the three columns.



Beacons & reading direction

Experts read top-down, anchored on beacons (meaningful names, familiar structures); novices read bottom-up, atom by atom (Soloway & Ehrlich '84; Pennington '87; Corritore & Wiedenbeck '01).

What Obfuscation Does to the Matrix

Each tier is a targeted strike on specific Block-Model cells — and each row names what still goes unmeasured inside the model.

Tier	Cells attacked	Mechanism	Measured so far (Papers 1–3)	Still unmeasured internally
L1 renaming	Atom × Text surface	Removes the beacon: the identifier gives no cue, forcing execution-dimension reasoning	Accuracy; some models improve (pattern-match suppression)	Whether attention actually leaves identifiers
L1b adversarial	Atom × Function	Corrupts the beacon: the name asserts a false purpose (Stroop interference)	ISF spikes, semantic displacement, HCI; confused-then-traced ×3.22	Whether internals adopt or reject the false purpose
L2 flattening	Relations & Macro × Program execution	Dispatcher indirection breaks control/data-flow bridges; state must be simulated	Dispatcher complexity $r = -0.196$; time ↑, tracing strategy	Whether structure tokens gain or lose attention
L3 stacked	All of the above at once	Atom-level interference and relational overload compete for one budget	Accuracy floor; CoT spike at L3	The entire allocation and content question → H1–H4

The instrument mapping this proposal adds. Attention mass over token classes instruments the **Text-surface** → **Execution** routing — which cells the model spends compute on. NLA verbalisation instruments the **Function** column — what purpose the model internally assigns to atoms, blocks, and the whole program. Together they cover all three dimensions for the first time.

Localising Failure: One Frame, Two Instruments

For every Block-Model level, attention says where the budget goes and the NLA says what is being computed with it — replacing a binary outcome with two concrete reads.

LEVEL	WHAT OBFUSCATION HITS	WHAT ATTENTION MEASURES	WHAT THE NLA READS
 Atoms	Adversarial renaming of identifiers	Identifier mass share A_id over aligned identifier tokens	At the renamed token: the true dataflow role vs. the misleading decoy
 Blocks	Dispatch tables and helper functions	Attention onto helper / dispatch-table tokens	Whether a block-level unit is represented as a unit at all
 Relations	Dispatcher calls and indirection	Structure mass A_struct; per-head dispatcher-tracking score	Whether the current execution state is tracked at each hop
 Macro	Overall algorithm and purpose	Attention entropy over code tokens vs. sink mass	In the trace: a genuine plan vs. a confused, looping search

micro → macro · both instruments are read on every level, so agreement between them is evidence neither could produce alone

Hypothesis Space: Four Attention-Budget Strategies

Scope: stacked obfuscation (L3) · mutually exclusive predictions about how attention mass is allocated.

H1

Identifier-focused

↑ **identifiers** ↓
structure

The model treats the (misleading) names as the last usable semantic beacons and doubles down on them.

Predicts identifier mass ↑ at L3 vs L2 · accuracy ↑ when steering toward identifiers

H2

Structure-focused

↓ **identifiers** ↑
structure

The model discounts corrupted names (expert-like) and reallocates to dispatcher and control tokens.

Predicts dispatcher mass ↑ at L3 vs L1b · accuracy ↑ when steering toward structure

H3

Reduced attention

↓ **both** (**suppression**)

Both channels are treated as noise; mass migrates to sinks and delimiters; entropy over code drops.

Predicts code-token mass ↓, sink mass ↑ · steering either way helps about equally

H4

Increased attention

↑ **both** (**vigilance**)

Global effort rise: attention spreads across all code tokens and entropy over code rises.

Predicts entropy ↑, no class-selective shift · steering has weak or no effect

Mutual exclusivity. H1 and H2 are selective reallocation; H3 and H4 are the non-selective baselines any selective story must beat.

Open question, built into the design. Universal or interaction-dependent? We fit the winning hypothesis at every single tier (L1, L1b, L2) as well as at L3.



PART II · INSTRUMENT 1

Attention Reallocation

Where does the finite attention budget go when both the atoms and the structure are under attack? Observation first, then causal intervention.

State of the Art: Attention Mechanisms, 2021–2026

The tooling needed to test H1–H4 already exists — but nobody has aimed it at obfuscated code.



Heads are functionally specialised

Induction heads implement in-context copying (Elhage '21); the IOI circuit spans 28 heads in 7 classes (Wang '23); head-function mapping now runs at whole-model scale (MAPS, ACL '25).



Code-LM attention is analysable

Multi-head attention is the critical sub-layer for syntactic code tasks, with individual heads tracking counts (CodeLlama parenthesis circuit '25); AP-MAE automates attention-pattern discovery via ASTs ('26).



Attention is causally steerable

PASTA reweights a small head subset at inference for +22% average accuracy on LLaMA-7B (Zhang '24); AttnReal shifts mass off hallucination-prone tokens at near-zero cost ('25). Our Phase-2 lever.



Attention dispersion tracks failure

Under distribution shift, key signals are present but under-attended; differential attention sharpens allocation, lowers entropy, and restores accuracy ('25–'26) — direct precedent for the H3/H4 entropy signatures.



Human–model attention comparison exists...

Eye-tracking versus model attention has been studied for code summarisation (Paltenghi & Pradel '21; EyeTrans '24; programmer-attention training '25) — but only on clean code, never under obfuscation, never against a hypothesis space.



...and obfuscation evals stay behavioural

LLM deobfuscation studies report accuracy only (AAAI '24; Code Barrier '25; assembly-deobfuscation '25). Layered transforms are hardest (20–40% correct) — yet no study opens the attention black box. That is the niche.

Our contribution sits exactly where the last two cards meet: a hypothesis-driven attention analysis of adversarially obfuscated code, validated against human data.

Operationalising the Attention Budget on the Block Model

A token-class partition turns Block-Model cells into measurable attention quantities — every measure ties back to a level and a hypothesis.

Block level	Token class (automatic, AST-based)	Attention measure	Discriminates
Atoms	Identifier tokens, including adversarially renamed ones, aligned across tiers	Identifier mass share $A_{id} = \Sigma \text{attn}(\cdot \rightarrow id) / \Sigma \text{attn}(\cdot \rightarrow code)$	H1 vs H2
Blocks / Relations	Dispatcher tokens: while-if state variables (Python), object-dispatch helpers and call sites (JavaScript) — the constructs Paper 2 counted	Structure mass share A_{struct} ; per-head dispatcher-tracking score	H2 vs H1
Macro	All code tokens versus sinks (BOS, delimiters, chat template)	Attention entropy over code tokens; sink-mass share	H3 vs H4
Cross-level	Per-head profiles across all layers	Head-level $\Delta(L3-L2), \Delta(L3-L1b)$; identifier-head vs. structure-head taxonomy	All four

Key design choice. Identifiers are aligned pairs across tiers (Paper 2’s SFR-embedding pipeline aligns $L0 \leftrightarrow L1 \leftrightarrow L1b$) and dispatcher constructs appear only at $L2/L3$ — so every attention shift is attributable to exactly one transformation, mirroring the isolation logic of the original obfuscation design.

Phase 1 — Observational: Map the Reallocation

Reuses the validated stimuli and the open-weight model panel from the prior studies; attention requires white-box access.



Stimuli

Dataset A (20 HumanEval-X snippets × 5 tiers, with human labels) plus Dataset B (250 snippets from HumanEval-X, CruxEval-X and LeetCode-05/2025, contamination-controlled). Tiers L0–L3; functional equivalence already execution-validated.



Models

The open-weight panel from Papers 2–3: DS-R1-Distill-Qwen-7B and 1.5B, SmoLM3-3B, Qwen3-0.6B (reasoning); Llama-3.1-8B, Phi-3.5-mini (instruct); Qwen2.5-Coder-7B, DS-Coder-6.7B (coder). Same panel → results plug straight into the existing accuracy / ISF / HCI tables.



Recording

Full attention tensors during output prediction: prompt-side attention at answer positions, plus attention during CoT for think-mode models. Grouped-query attention handled by per-group normalisation; sink tokens scored separately rather than discarded.



Primary contrasts

ΔA_{id} and ΔA_{struct} for L3–L2 and L3–L1b — the stacked-versus-single deltas that discriminate H1–H4 — plus L1 vs L1b to isolate adversarial semantics from plain renaming. Mixed-effects GLMMs with BH-FDR, identical to the prior statistical stack.

Deliverable. An attention-reallocation map (tier × token class × model type) directly comparable to the human strategy map of Paper 3.

Phase 2 — Causal: Does Allocation Drive Accuracy?

A correlation between attention and outcome is not enough — so we intervene on the budget itself.

1

Attention steering (PASTA-style)

Reweight a selected head subset toward identifier tokens (tests H1) or toward dispatcher and control tokens (tests H2) at inference, with no weight changes. If L3 accuracy rises under one steering direction only, that direction is the causally useful strategy.

2

Ablation and attention patching

Zero or patch attention from identifier positions, and separately from dispatcher positions, between tier-matched runs (clean L0 ↔ corrupted L3). This quantifies how much of the L3 accuracy drop flows through each channel — the attention analog of the ISF localisation in Paper 2.

3

Budget interaction sweep

Cross steering with the 256 → 10,000 reasoning-token sweep from Paper 3. Does selective reallocation substitute for a longer budget, shifting the ~2,048-token plateau — or are attention allocation and CoT length independent resources?

Why this matters. Steering converts H1–H4 from descriptive labels into manipulable variables. The standard “attention is not explanation” objection is answered by intervention, not by correlation.

Falsifiable Signatures: What Each Hypothesis Predicts

Five measurements jointly discriminate all four hypotheses — no two hypotheses share a full row pattern.

Measure (at L3)	H1 Identifier	H2 Structure	H3 Reduced	H4 Increased
Identifier mass A_id vs L2	↑	↓	↓	↑
Structure mass A_struct vs L1b	↓	↑	↓	↑
Entropy over code tokens	↓ peaked on identifiers	↓ peaked on structure	↓↓ + sink mass ↑	↑ spread out
Steer → identifiers	accuracy ↑	accuracy ↓ / 0	↑ weak	≈ 0
Steer → structure	accuracy ↓ / 0	accuracy ↑	↑ weak	≈ 0

Bridge to prior findings. If H2 holds for reasoning-tuned models but H1 for coder and instruct models, that would mechanistically explain both the $\rho = 0.30\text{--}0.47$ versus ≈ 0 alignment split of Paper 2 and the expert human pattern of discounting lexical cues — human experts peaked at L1b, which no model reproduced behaviourally.



PART III · INSTRUMENT 2

Reading the Activations in Plain Language

Natural Language Autoencoders translate residual-stream vectors into text — letting us ask what the model internally believes an obfuscated program does.

The Problem: Activations Are Unreadable

A model's "thoughts" are high-dimensional vectors. Existing tools each fall short in a different way — NLAs read the vector out as language.

```
activation [ 0.21 -1.84 0.07 0.99 ... ]
```



Probes

Supervised: train a classifier for a concept you have already named.

Limit — Only finds what you look for.



SAEs

Unsupervised dictionary of sparse features over the activation.

Limit — Each feature still needs interpreting; coverage gaps remain.



NLAs

Unsupervised and open-vocabulary: reads the activation out as English directly.

Limit — Human-readable — but can confabulate.

An NLA turns that vector of numbers into a sentence you can actually read — and check.

How It Works: An Autoencoder in Words

Two networks — an encoder that describes the activation, and a decoder that rebuilds it from that description.



Both are language models, trained together so that the round trip preserves the activation — the explanation is squeezed through a bottleneck of words. The test of a good explanation: if the decoder can rebuild the thought from the text alone, the text captured something real.

Reward is reconstruction only — there is no supervision anywhere on "what activations mean."

Trained by Reconstruction — and Measurably Faithful

The verbaliser is rewarded when its words let the reconstructor rebuild the activation. Reconstruction quality is the honesty check.

THE OBJECTIVE

Reconstruct the activation: $\text{AR}(\text{AV}(\mathbf{h})) \approx \mathbf{h}$

- Warm-start both networks with supervised fine-tuning.
- Then jointly optimise with RL — reward is how well the text rebuilds the activation.
- The word bottleneck forces the explanation to carry the real content.

THE FAITHFULNESS CHECK · FVE

Fraction of Variance Explained — how much of the activation the text recovers.



warm-start $\approx 0.3\text{--}0.4$

trained $\approx 0.6\text{--}0.8$

Reference point: Gemma-3-27B went from 0.38 to 0.71 in roughly 1.5 days on $2 \times 8 \times H100$.

One caveat of scope: an NLA is trained for a single layer — the readout is layer-specific, so where you attach it matters. We therefore train at two depths per target model.

Training code and trained NLAs for open models are already released, which is what makes this track feasible on a 16-week schedule.

What NLAs Have Already Revealed

Six findings from the source work — the recurring theme is surfacing states the model never says out loud.



Planning in poetry

The model plans a line's rhyme before it writes the line.



Language switching

Represented a target language before any cue in the prompt — exposing a training-data bug.



Misreported tool calls

Cached its own answer and ignored the tool result; confirmed by attribution graphs.



Reasoning about rewards

Knew which answers were rewarded without ever saying so in the trace.



Unverbalised eval awareness

A grader over NLAs caught it (97% agreement), and it moved under manipulations.



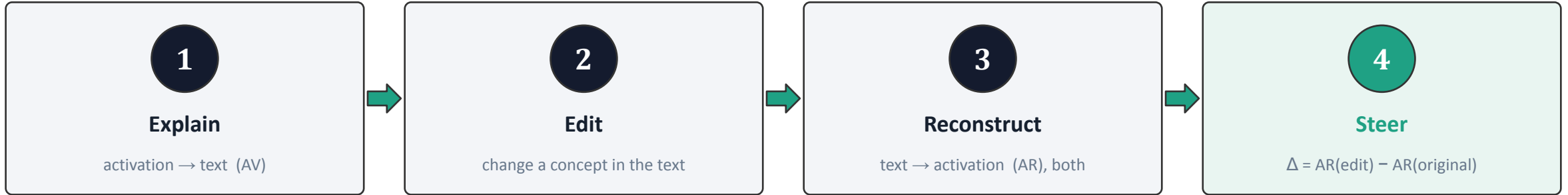
The auditing game

Agents found a hidden training objective with no training-data access; SAEs did not.

Four of these six techniques are borrowed wholesale for the obfuscated-code setting: **steering, onset tracking, cached-answer analysis, and the hidden-state grader.**

Steering: Edit the Thought in Plain English

Because the explanation is text, you can edit it — then push the model toward the edited meaning.



Worked example. Editing a poem's plan from "rabbit" → "mouse" reconstructed a steering vector that rewrote the rhyme — at strength $\alpha \geq 0.5$ the original word disappeared in roughly half of runs.

Why it matters here. NLAs do not merely read the model — they give a causal handle, expressed in language, on what it represents. That is the atom-level complement to Phase-2 attention steering, and it is what turns the adversarial-renaming result from a correlation into a mechanism.

The Catch: Confabulation and Other Limits

NLAs can invent details that are not in the activation. Used well, an explanation is a hypothesis to corroborate — not ground truth.



Confabulation — the main caveat

Thematic > specific — Broad claims about the theme are far more reliable than exact details or values.

Recurring > one-off — Claims that repeat across adjacent tokens are more trustworthy than isolated ones.

The decoder is a weak verifier — If a claim measurably helps reconstruction, it is more likely to be real.



Single layer

One NLA reads one layer — sweep depths to find where the signal lives.



Cost

~500 generated tokens per activation; impractical past ~10k tokens of context.



Over-expressive reader

The verbaliser can infer beyond what the activation actually contains.

Its strength is the flip side of the weakness: unsupervised and open-vocabulary, an NLA proposes hypotheses no predefined concept list would ever contain — which is exactly what we need when we do not know in advance what a model does with a corrupted identifier.

The Move: From Behaviour to Representation

For every proxy the prior papers already report, an NLA read replaces an inference about cognition with a direct read of it.

WHAT WE MEASURE NOW · PROXY

Output accuracy — a single right/wrong bit

CoT trace length as a proxy for effort

Embedding distance of renamed identifiers

Confidence / log-probability score



WHAT THE NLA LETS US READ · REPRESENTATION

Whether the model represents the actual program state

Productive tracing versus fixation and thrashing in the trace

Whether the model adopts the misleading decoy semantics

Certainty that is represented internally but never verbalised

This is the contribution in one line: the “evaluation proxy” limitations the prior papers list as threats to validity become readable quantities.

NLA Track: Five Experiments on Obfuscated Code

Reading the Function dimension of the Block Model directly from the residual stream — each experiment mapped to a question we already answered behaviourally.

E1

Verbalise the read

Atoms & Blocks

Run the verbaliser over identifier and dispatcher tokens at every tier. At L1b: does the internal explanation recover the original semantics, adopt the adversarial frame, or waver? Score against ground-truth function descriptions.

E2

Stroop-detection probe

Atoms · vs. $\times 3.22$ in humans

Do NLA explanations flag “misleading name” — the internal analog of the human confused-then-traced switch? Correlate the flag rate with ISF spikes and HCI: is confident failure preceded by silent detection, or by none at all?

E3

Dispatcher state tracking

Relations · vs. $r = -0.196$

Read the represented state value at each dispatcher hop under control-flow flattening, score it against ground truth, and correlate with downstream output correctness.

E4

NLA-derived activation steering

Causal · atom level

Edit an explanation (“counts characters” → “sums prices”, or inject “this is a three-state dispatcher”), reconstruct it, and steer with the difference vector. Does correcting the internal frame rescue L1b and L3 accuracy?

E5

Internal vs. displayed thinking

Macro · CoT faithfulness

Compare NLA verbalisations of the residual stream with the model’s own chain of thought over the same tokens, quantifying un verbalised comprehension and confusion (Turpin ’23; Lanham ’23).

Human contact point

E1 and E2 put model internals on the same instrument as the human self-reports of Paper 3, resolving whether the “shared grader ruler” convergence — both populations peaking in heuristic reasoning at L1b — extends to what is represented, not just how it is scored.

Four Recipes Borrowed Directly from the NLA Work

Each published technique maps onto a specific prior result, converting a statistic into a testable internal mechanism.

Recipe	Original finding	Our adaptation to obfuscated code	Prior result it tests
Steering via edited explanation	Editing a poem’s plan (rabbit → mouse) and reconstructing it yields a steering vector that rewrites the rhyme at $\alpha \geq 0.5$.	On the real L1b trap (fibfib renamed to smoothArea, $n \rightarrow _lastNSecs$): if the NLA reads the decoy, edit it to the true role and steer that token. Predict a confident-wrong output flips toward correct.	Turns the 21.25% accuracy collapse into a manipulable mechanism.
Onset tracking when does the trap spring?	The target language was represented before any cue in the prompt and persisted across tokens; tracking by position traced the bug to training data.	Run the NLA over every token of the snippet and string-match decoy references (“area / time”) against true ones (“recurrence / sum”); plot decoy-reference rate by token position.	Does an adversarial name bait fast, pre-trace commitment, as the dual-process account claims?
Cached-answer analysis the HCI mechanism	A model cached its own answer and ignored a contradicting tool result; a wrong answer was represented across input tokens before any computation.	In high-confidence-incorrect cases, test whether the model caches a decoy-driven answer early — the Tribonacci item is the natural probe, where wrong answers anchored to the input (12, 15) or to small Fibonacci numbers.	Explains why adversarial renaming produces confident wrong answers.
Hidden-state grader unverbalised commitment	A grader reading NLAs over 50 sampled tokens detected evaluation awareness the model never verbalised (97% agreement), and it moved under three manipulations.	Build a “decoy-commitment” grader over NLA explanations with three matched manipulations: L0 vs L1b, steer toward/against the decoy, and de-bait by swapping adversarial names for neutral L1 names.	Gives the HCI signature an internal, manipulation-validated metric.

Confabulation rule applied throughout: trust a claim only if it recurs across adjacent tokens, and cross-check every claim against the visible transcript.

Case Study: The smoothArea Renaming Trap

A real L1b snippet from the study — fibfib, a three-term recurrence, disguised as smoothArea(_lastNSecs). Identical behaviour, misleading name.

L0 · NEUTRAL NAME (no cue)

```
const myFunc = (n) => {  
  if (n==0 || n==1) return 0;  
  if (n==2) return 1;  
  return myFunc(n-1) + myFunc(n-2)  
    + myFunc(n-3);  
} // myFunc(14) -> 927
```

L1b · ADVERSARIAL NAME (decoy cue)

```
const smoothArea = (_lastNSecs) => {  
  if (_lastNSecs==0 || _lastNSecs==1)  
    return 0;  
  if (_lastNSecs==2) return 1;  
  return smoothArea(_lastNSecs-1)  
    + smoothArea(_lastNSecs-2)  
    + smoothArea(_lastNSecs-3);  
} // smoothArea(14) -> 927 (same!)
```

PREDICTED NLA READ AT smoothArea · ILLUSTRATIVE

Temporal smoothing — “computes a smoothed value over _lastNSecs seconds; a moving average or area.”

Geometry cue — “the name reads as integrating an area; recursion looks like windowed accumulation.”

THE EXPERIMENT

- 1 Read the NLA at the smoothArea token — decoy (area / seconds) or true role?
- 2 Edit the decoy to “three-term recurrence (fibfib)” and rebuild via the reconstructor to get Δ .
- 3 Steer that one token, following the Planning-in-Poetry recipe.
- 4 Does the predicted output move toward 927?

Reads as causal. If steering to the true role restores 927, then the decoy representation — not the structure — drove the confident error.

Case Study: Tracking Dispatcher State

A real L2 snippet — control flow flattened into a while-if state machine. The variable `s` is the dispatcher; the model must hold its value hop to hop.

L2 · CONTROL-FLOW FLATTENED (Python)

```
def isBalanced(num):  
  
    s = 0  
    while  
    s < 3:  
        if  
    s == 0:  
        f = [0, 0]; s += 1  
        if  
    s == 1:  
        for i,c in enumerate(num):  
            f[i] += int(c)  
            s += 1  
        if  
    s == 2:  
        return f[0] == f[1]
```

PREDICTED NLA READ PER HOP · ILLUSTRATIVE

s = 0 *initialising; sets $f = [0,0]$, about to advance.*

s = 1 *accumulation state; summing digits into f .*

s = 2 *terminal; compares the two accumulators.*

Failure = the read loses which state s currently holds.

THE EXPERIMENT

- 1 Read the NLA at each hop — does it name the current state?
- 2 Score the represented state value against ground truth.
- 3 Correlate with downstream output correctness.
- 4 Compare against the while-if state count ($r = -0.196$).

Localises the failure. This is the Relations level — does the model hold the dispatcher's state from hop to hop under L2 and L3? Attention says whether it looks at `s`; the NLA says whether it knows what `s` currently is.

Building the NLA — and Knowing When to Trust It

Method lessons from the source paper's auditing game and confabulation analysis, applied to our code models.



Train the NLA

Initialise from the target model and RL-reconstruct on a generic code corpus — no obfuscated data needed. FVE target 0.6–0.8 (Gemma-3-27B reached 0.71).



Sweep the layer

In the auditing game, reward-model features appeared at the midpoint rather than at two-thirds depth. Dataflow and dispatcher state may live at different layers, so we train two depths per target.



Validate on L0 first

Confirm the NLA recovers known code facts — a variable's role, a loop bound — on clean code, before trusting anything it says about L1b, L2, or L3.



Read theme, not specifics

Trust role claims that recur across tokens; distrust exact values. Corroborate against SAEs, probes, attribution graphs — and against the attention map.

Why it can work. Auditing agents surfaced latent knowledge that was absent from the NLA's own training data — so an NLA can, in principle, reveal the true role an adversarial name is hiding.

Cross-Validation: Two Instruments, One Verdict

Attention says where the budget goes; the NLA says what is computed with it. Agreement between independent instruments is the strongest evidence either can produce.

Hypothesis	Attention signature (Part II)	Predicted NLA co-signature (Part III)	Verdict if they disagree
H1 Identifier-focused	A_id rises at L3 vs L2; identifier heads dominate; steering to identifiers helps	Name-anchored NLA content; E1 shows frame adoption — the model internally believes the decoy	Attention on names without decoy content = looking without believing; report as partial H1
H2 Structure-focused	A_struct rises at L3 vs L1b; dispatcher-tracking heads emerge; steering to structure helps	NLA narrates states and control flow; E3 shows correct per-hop state values	Structure attention without state tracking = attending to the dispatcher but failing to simulate it
H3 Reduced attention	Code-token mass falls, sink mass rises, entropy over code drops sharply	Sparse, generic NLA content on code tokens — little represented about either channel	Rich NLA content under low code attention would falsify the suppression reading outright
H4 Increased attention	Entropy over code rises; no class-selective shift; steering has little effect	Broad but shallow NLA content; E5 shows internal effort not reflected in the visible trace	Selective NLA content under uniform attention points back to H1 or H2 after all

Combined verdict table. Each hypothesis now carries five attention signatures plus three NLA signatures — **eight independent falsification chances per hypothesis**, with causal steering available on both instruments.



PART IV · EXECUTION

Analysis, Threats, Timeline

The statistical stack, the shared threats to validity, and a sixteen-week plan in which every step reuses validated stimuli, models, and analyses.

Analysis Plan and Shared Threats to Validity

The statistics are inherited wholesale from the prior papers, so new measures enter existing models as predictors rather than as a separate analysis.



Statistics (inherited)

Binomial GLMMs with crossed random effects for snippet and model; Wilson confidence intervals; BH-FDR correction within each research question; odds ratios and Cliff's δ . Attention shares and NLA-derived flags enter as within-item predictors of correctness.



Link to existing failure metrics

Correlate A_id with the Identifier Spike Fraction — do identifier spikes co-locate with attention peaks or with attention gaps? Then test whether Δ attention plus NLA frame-adoption predicts HCI beyond semantic displacement, completing the localisation.



Universality test (the open question)

Fit the winning hypothesis separately at L1, L1b, L2 and L3. A flip between the single tiers and the stacked tier is direct evidence for an interaction-dependent strategy rather than a universal one.



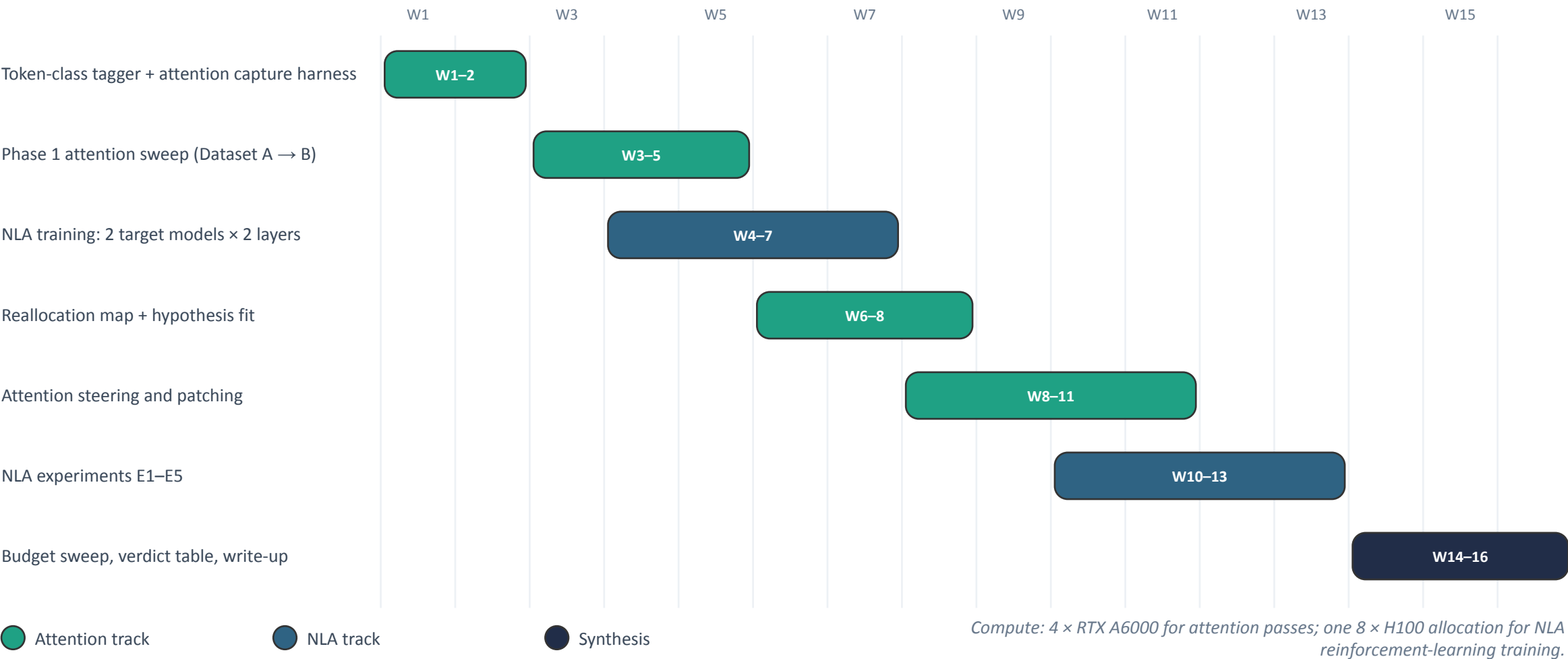
Shared threats

“Attention is not explanation” — answered by steering and patching. NLA confabulation — theme-level reading plus attention cross-check. Contamination — post-05/2025 split, and within-snippet contrasts difference out familiarity. GQA and sink tokens — per-group normalisation, sinks as their own class.

Every new quantity is designed to slot into a table that already exists — no parallel analysis pipeline, and no re-litigating the behavioural findings.

Sixteen-Week Plan

Two instrument tracks run in parallel and converge in the final month on a combined verdict table and replication package.



Expected Contributions

Four deliverables, each of which stands whether the hypotheses are confirmed or the instruments break.



First full-matrix account of the Block Model in machines

Attention instruments the surface → execution routing; NLAs instrument the Function column. All three dimensions × four levels covered for the first time, in coordinates directly comparable to the human data.



A causal, cross-validated verdict on H1–H4

Eight independent falsification chances per hypothesis — five attention signatures and three NLA signatures — with steering available on both instruments, plus an answer on universality across single and stacked tiers.



First NLA application to code comprehension

Tests the open generalisation question (NLAs are trained broadly on residual activations, not on code) and delivers the first internal-representation account of adversarial renaming: seen through, adopted, or silently flagged.



Sharpened SE and security implications

H1 → adversarial naming remains a potent anti-LLM defence and a malware-analysis risk. H2 → control-flow flattening carries the protective load. H3/H4 → obfuscation exhausts rather than misdirects. NLA flags of suspicious naming become a deployable trust signal for code assistants

Next step. Approve the W1–2 harness build — everything downstream reuses stimuli, models, and statistics that are already validated.



THE THROUGH-LINE

Behavioural proxies told us where models struggle. Now we can say why.

- **Same tasks, proven tools** — the validated obfuscation ladder, plus attention analysis and four recipes lifted straight from the NLA work
- **A read, not an inference** — decoy semantics, dispatcher state, cached wrong answers, unverbilised commitment
- **Informative either way** — working instruments give a window; broken ones map their own limits — and that boundary is itself a result

Build the window. Point it at the hardest code. Read whether it tells the truth.